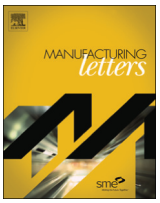
Available online at www.sciencedirect.com

Manufacturing Letters

Manufacturing Letters 41 (2024) 1246–1256



52nd SME North American Manufacturing Research Conference (NAMRC 52, 2024)

Deep reinforcement learning based proactive dynamic obstacle avoidance for safe human-robot collaboration

Wanqing Xia[a,*], Yuqian Lu[a], Weiliang Xu[a], Xun Xu[a]

[a]*The University of Auckland, 20 Symond Street, Auckland 1010, New Zealand*

Abstract

Ensuring the health and safety of human operators is paramount in manufacturing, particularly in human-robot collaborative environments. In this paper, we present a deep reinforcement learning-based trajectory planning method for a robotic manipulator designed to avoid collisions with human body parts in real-time while achieving its goal. We modelled the human arm as a freely moving cylinder in 3D space and formulated the dynamic obstacle avoidance problem as a Markov decision process. The algorithm was tested in a simulated environment that closely mimics our laboratory environment, with the goal of training a deep reinforcement learning model for autonomous task completion. A composite reward function was developed to balance the effects of different environmental variables, and the soft-actor critic algorithm was employed. The trained model demonstrated a 93% success rate in avoiding dynamic obstacles while achieving its goals when tested on a generated data set.

© 2024 The Authors. Published by ELSEVIER Ltd.
This is an open access article under the CC BY-NC-ND license
(http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the scientific committee of the NAMRI/SME.

Keywords: *Human-robot collaboration; Dynamic obstacle avoidance; Deep reinforcement learning; Reward engineering*

1. Introduction

Robotic manipulators are increasingly becoming integral components of the manufacturing industry worldwide, enhancing production efficiency and enabling the mass production of products at more affordable prices. Research indicates that integrating the adaptability and flexibility of humans with the precision and endurance of robots can result in a level of productivity that surpasses what could be achieved by either human or robotic systems alone [19]. To actualise cost-competitive manufacturing through human-robot collaboration, it is imperative to remove the physical barriers that currently separate humans and robots in future smart manufacturing environments. This will facilitate closer interactions between humans and robots [24]. However, it is crucial that the safety of human operators is not compromised in the pursuit of higher productivity. This necessitates advancements in reliable 3D perception, high-level planning, and sophisticated control within manufacturing systems [14]. Specifically, for human-robot collaborative manipulators, it is vital to have a dependable environment perception system capable of detecting and locating the human body among other potential obstacles in the workspace. Additionally, these manipulators must possess the decision-making capabilities required to complete tasks in constrained environments. In Fig 1, we showcase a human-robot collaborative assembly scenario set up in the Laboratory for Industry 4.0 Smart Manufacturing Systems (LISMS) at the University of Auckland. The UR5e robot featured in this scenario is outfitted with an RGB-depth (RGB-D) camera for environmental perception, a force and torque sensor, and a parallel gripper to facilitate task completion and handover to the human operator. Importantly, the UR5e robot is controlled by a desktop computer equipped with a high-performance graphics card, necessary for running sophisticated artificial intelligence control algorithms.

* Corresponding author. Tel.: +64-0212993410
E-mail address: wxia612@aucklanduni.ac.nz (Wanqing Xia).

2213-8463 © 2024 The Authors. Published by ELSEVIER Ltd.
This is an open access article under the CC BY-NC-ND license
(http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the scientific committee of the NAMRI/SME.



Figure 1. A human-robot collaborative assembly scenario in LISMS laboratory

Currently, collision-free human-robot collaboration can be facilitated using traditional trajectory planning algorithms supplemented by a set of sensors. While a robot's state is readily accessible, sensors predominantly function to track human movement. The robot modifies its behaviour in response to human actions based on its state–an approach referred to as reactive trajectory planning [9]. Due to its inherently responsive nature, the robot is frequently required to decelerate or, in some instances, halt to mitigate potential impacts when in proximity to humans. Such systems, although simple to implement, ensure safety in human-robot interactions. Nonetheless, the introduction of slowed movements and increased idle times for the robot can significantly diminish manufacturing efficiency. Accordingly, there is a compelling need for a trajectory planning algorithm that preserves the safety of human-robot collaboration without sacrificing operational efficiency.

To address this, a shift from reactive to proactive trajectory planning is advocated. Proactive systems, taking into account the dynamic nature of human movement throughout the manufacturing process, can forecast an operator's future actions and delineate a collision-free path, thereby obviating the need for the robot to decelerate or stop [13]. Traditional algorithms, which typically map a fixed global path from start to finish, lack the necessary adaptability to account for deviations in the human operator's predicted actions. Recently, reinforcement learning has gained traction as a robust approach in trajectory planning. By enabling robots to learn tasks through trial and error, it minimises the need for algorithmic redesign or fine-tuning across different environments [31]. Furthermore, reinforcement learning algorithms operate by assessing the current state of the system to generate subsequent actions. This negates the need for extensive prediction of human actions and allows for local trajectory adjustments, aligning well with the proactive planning requirements of human-robot collaborative manufacturing.

Recent advancements in robotics have witnessed the increasing incorporation of deep reinforcement learning (DRL) into the trajectory planning of robotic arms, signifying a paradigm shift in both the complexity of tasks that robots can perform and the efficiency with which they execute them. Designing an appropriate reward function is one of the main challenges in reinforcement learning [10]. The reward function serves as a compass guiding the robotic agent through the intricate landscape of possible actions towards the desired behaviour. A meticulously crafted reward function can remarkably accelerate the learning process, enabling robots to carry out tasks with high precision. Another critical aspect of leveraging DRL in robotic trajectory planning is the challenge of the simulation-to-real (sim-to-real) transfer [11]. This involves bridging the gap between the model's performance in a controlled simulation and its application in a real-world scenario–a leap that often presents significant hurdles due to the unpredictable nature of real-world dynamics. Efforts in this domain have concentrated on developing robust algorithms capable of generalising the training received in simulation to operate seamlessly when deployed, thus mitigating the risks and costs associated with direct real-world training.

In our prior research, we demonstrated the efficacy of the Soft Actor-Critic (SAC) algorithm in the context of robotic manipulator trajectory planning in scenarios devoid of moving obstacles[32]. In the current study, we involve free-moving obstacles into the environment, addressing the challenge of proactive trajectory planning through the application of the same algorithm. We propose a novel reward structure that effectively directs the robot to achieve its target without collisions in an environment populated with dynamic obstacles. Additionally, we introduce a methodology that facilitates the transition from simulation to real-world application, mitigating the sim-to-real transfer problem. This approach enables the safe training of robot manipulators in simulated settings, safeguarding against potential damage to both the robot and human operators resulting from unforeseen actions.

The remainder of this paper is organised as follows: Section 2 provides a review of literature pertinent to this study, Section 3 details the DRL algorithm and the specifics of our model design. Section 4 introduces our reward function design and its integration with the current algorithm. Experimental results and associated discussions are presented in Section 5. Finally, Section 6 summarises the conclusions of this work and proposes directions for future research.

2. Related Work

Safety is the paramount concern in human-robot collaborative manufacturing, necessitating contributions from multiple domains including sensing technology, trajectory planning algorithms, and robot control techniques. This section begins with a review of prior work pertinent to safety in human-robot collaboration with traditional methods. Subsequently, it delves into reinforcement learning algorithms and then explores collision-free trajectory planning based on reinforcement learning.

2.1. Safe human-robot collaborative manufacturing with traditional methods

Efficient and safe collaboration with robots has always been a central research problem, captivating the interest of many in the field. Various methods have been proposed to address this challenge. For instance, the survey paper by Villani et al. [30] offers an extensive review of human-robot collaboration in industrial environments, highlighting safety, intuitive user interfaces, and applications. It identifies key areas where HRC is particularly beneficial, including handling, welding, assembly, and the automotive industry.

Further advancing this field, Lasota et al. [12] proposed a framework for adaptive and risk-aware robot motion planning and control in close proximity HRC. This framework models the uncertainty in human actions using the entropy rate in a Markov Decision Process (MDP), integrating this model into simulations with human workers and robotic assistants. Polverini et al. [22] introduced a novel approach for real-time collision avoidance in human-robot interaction, focusing on a kinetostatic safety field that considers both the relative position and velocity between a robot and a human. This method enhances risk evaluation in close proximity interactions. In 2016, Lo et al. [18] proposed virtual impedance control operating in a risk space, accounting for potential collision points and adjusting the robot's dynamics based on a risk function influenced by proximity to obstacles, including humans.

The integration of depth sensors into research, as discussed by Ragaglia et al. [23], has enabled the creation of a more comprehensive and accurate representation of the human worker's position and movements within a robotic workspace, thus enhancing HRC safety and efficiency. Casalino et al. [1] introduced a proactive approach to robotic path planning in industrial HRC scenarios, anticipating potential human activities and adjusting the robot's path in advance to minimise collision risks. Finally, Rubagotti et al. [25] employed a Model Predictive Control (MPC) approach for robot motion control, optimally re-planning at each sampling time and considering a sequence of control moves within a given prediction horizon via numerical optimisation. These diverse approaches collectively underscore the rapid evolution of HRC, emphasising safety and efficiency through innovative technological integration and anticipatory strategies.

2.2. Reinforcement learning algorithms

The field of reinforcement learning has witnessed significant advancements owing to a variety of algorithms, each uniquely addressing specific challenges and thereby propelling the evolution of the discipline. These algorithms can be broadly classified into two main categories: off-policy and on-policy methods.

On-policy methods comprise strategies such as Trust Region Policy Optimization (TRPO) [27], Asynchronous Advantage Actor-Critic (A3C) [20], and Proximal Policy Optimization (PPO) [28]. TRPO introduces an approach for optimizing control policies with a guaranteed monotonic improvement, proving particularly effective for large nonlinear policies like neural networks. A3C, on the other hand, demonstrates the stabilising influence of parallel actor-learners in training, facilitating the successful implementation of neural network controllers. This method surpasses other algorithms in the Atari domain, notably requiring only half the training time. PPO addresses the complexities and instabilities of previous policy gradient methods by optimizing a novel objective function using stochastic gradient ascent, offering practical solutions to problems inherent in earlier RL algorithms. However, on-policy algorithms have a limitation: the state, action, and reward information are discarded after each training process, often necessitating a larger number of training examples compared to off-policy methods.

Off-policy methods leverage replay buffers during algorithm training, storing information about actions, state changes, and reward feedback for future use. This category includes Deep Q-Network (DQN) [21], Deep Deterministic Policy Gradient (DDPG) [16], Normalized Advantage Functions (NAF) [7], Soft Actor-Critic (SAC) [8], and Twin Delayed Deep Deterministic policy gradient (TD3) [6]. The DQN algorithm, a pioneer in applying deep learning models to control policies derived from high-dimensional sensory input, outperformed all previous methods on several games and even exceeded human expert performance in three. DDPG adapts the DQN concept to continuous action domains and introduces an actor-critic framework using deep function approximators for policy learning in high-dimensional, continuous action spaces. NAF proposes a continuous variant of Q-learning, eliminating the need for a separate actor or policy function, resulting in a simpler, more sample-efficient algorithm, especially beneficial when using large neural network function approximations. SAC integrates the concept of a maximum entropy framework, encouraging broader policy exploration and acquiring diverse behaviours, thus enhancing learning efficiency and robustness. Lastly, TD3 tackles the issue of overestimation bias in actor-critic methods by introducing clipped double Q-learning, which employs two critics (value functions) and adopts the lower value to reduce overestimation. Furthermore, TD3's delayed policy updates minimize per-update errors and enhance performance, yielding a more stable learning process.

2.3. Trajectory planning with reinforcement learning

In the field of manipulator trajectory planning and control, a common environmental setup involves a fixed robotic manipulator learning to reach a target position or pose amidst moving obstacles. These research endeavours can be categorized based on the moving degree of freedom (DoF) of the obstacles. In the simplest scenario, the obstacle is confined to move periodically along a linear track [26] [4]. The low DoF of the obstacle simplifies the model, making it easier to train. In a more complex 2D setting, obstacles can move across the tabletop [3]. Some studies have elevated the obstacle to an aerial position, yet still restrict its movement to a planar domain [2] [5].

Our primary interest lies in scenarios where obstacles freely move in a 3D space, a setting that closely mirrors the actual work environment in HRC. Zindler et al. developed a dynamic

obstacle avoidance system using Deep Reinforcement Learning (DRL), adapting a robot's planned trajectory in real-time to circumvent dynamic obstacles [33]. They devised a multi-component reward function, each part targeting a specific desired behaviour. Li et al. introduced a hybrid approach that combines RL with the Artificial Potential Field (APF) method, surmounting the traditional APF's limitations in effectively avoiding both dynamic and static obstacles near targets [15]. In another notable study, Liu et al. proposed the Intrinsic Reward-DDPG method. They tackled the issue of static reward functions in training, proposing a reward system comprising both extrinsic and intrinsic rewards. The intrinsic reward function's parameters are updated through policy gradients to maximize the expected accumulated fitness [17].

3. Problem De nition

In this section, we define the problem of collision-free trajectory planning within the framework of a Markov Decision Process (MDP) and elucidate its relationship with deep reinforcement learning methodologies. Specifically, the Soft Actor-Critic (SAC) algorithm is adopted to address this problem. We delve into the mechanics of the SAC algorithm, detailing its application to our problem and discussing its efficacy in facilitating efficient learning for our model. At the end, we introduce our proposed framework for transferring the learned safe trajectory planning model from simulated environment to real world.

3.1. Markov decision process

Markov decision process is a mathematical framework for modelling decision-making in a partially controllable environment. MDP is used to find policies that specify the best action to take in a given state to maximise the long-term reward. It is the fundamental concept in reinforcement learning and is widely used for solving various problems in fields such as robotics, economics, manufacturing, and artificial intelligence [29]. A MDP can be defined by a tuple $< \mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P}, \gamma >$, which consists of a state set $\mathcal{S}$, an action set $\mathcal{A}$, a reward function $\mathcal{R}(s_t, a_t)$, a state transition function $\mathcal{P}(s_{t+1} \quad s_t, a_t)$, and the discount factor γ. An agent's behaviour is defined by a policy, π, which maps states to a probability distribution over the actions $\pi : \mathcal{S} \quad \mathcal{P}(\mathcal{A})$. The return from a state is defined as the sum of discounted future rewards:

$$\mathcal{R}_t = \sum_{i=t}^{T} \gamma^{(i-t)} r(s_i, a_i) \tag{1}$$

The goal of RL is to learn an optimal policy that instructs the agent on the best action to take in each state to maximise its long-term reward. The objective function $\mathcal{J}$ is often used to denote the performance of a policy in RL. It refers to the expected cumulative reward that an agent would receive starting from a state s and following a policy π, is given as:

$$J(\pi) = \mathbb{E}_{\pi}\left[\sum_{i=t}^{\mathcal{T}} \gamma^{(i-t)} R_i \, S_t = s\right] \tag{2}$$

In this study, our goal is to train the UR5e manipulator and let it learn to reach a goal pose in the 3D space autonomously by planning and executing a collision-free trajectory. The origin of the world frame is set at the bottom center of the UR5e manipulator, where the positive X, Y, and Z axis points toward the human operator, to the right-hand side of the human, and to the ceiling, respectively (The position of manipulator and human as shown in Fig 1. We use $\mathcal{P}_e = [p_e, \ \phi_e]^T \quad C^6$ to represent the end-effector's pose, where p_e being the three-dimensional position and ϕ_e being the Euler angle rotation in the Cartesian space related to the world frame, Fig 2(a). The joint angles $\Theta = (\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6)$ denotes the angle of each robot joint in radians. Thus, we can calculate the pose of the end-effector $\mathcal{P}_e$ at any given time t using forward kinematics: $\mathcal{P}_t = Forward_Kinematics\,(\Theta)$. Similar with the end-effector pose $\mathcal{P}_e$, the goal pose can be defined as $\mathcal{P}_g = \left[p_g, \ \phi_g\right]^T \quad C^6$,related to the world frame, shown in Fig 2(c). To move the manipulator to achieve the desired pose, the rotation of each robot joint needs to be controlled. Thus, our action space is defined as the change in angular position of each joint $\mathcal{A} = \Delta\Theta = \ (\Delta\theta_1, \Delta\theta_2, \Delta\theta_3, \Delta\theta_4, \Delta\theta_5, \Delta\theta_6)$. The next joint angles can be found by $\theta_{t+1} = \theta_t + \Delta\theta$.

To ensure dynamic obstacle avoidance without collision, it is imperative to monitor and observe the obstacle's state continuously. In our research, the premise is that all objects within the workbench area remain static except for the human operator's arm, which is the sole dynamic obstacle. Positioned on the opposite side of the human operator, the robot's domain is such that only the operator's hands and forearms intersect with the manipulator's operational zone. This allows for a simplification in our model, where the operator's hand and forearm are represented as a cylindrical dynamic obstacle. The dimensions of this cylinder—10 cm in diameter and 40 cm in length—mirror the approximate size of an adult's arm, shown in Fig 2(b). For locating this obstacle within three-dimensional space, we define its 6D pose as $\mathcal{P}_o = [p_o, \ \phi_o]^T$, where p_o and ϕ_o are the relative position and rotation of the obstacle's center of mass related to the world frame. Proactive trajectory planning requires not only the current position but also a prediction of the obstacle's future state. Therefore, our model incorporates both the positional data and the velocity of the obstacle, the latter denoted as $\mathcal{V}_o = [v_o, \ \omega_o]^T$, represents the linear and angular velocity of the obstacle's center of mass related to the origin.

When transitioning from a theoretical model to practical applications, we must establish two critical assumptions. The first is that the position and velocity of the obstacle can be measured with high accuracy by the sensing apparatus. The second assumption stipulates that the acceleration of the obstacle remains relatively consistent between consecutive measurements;

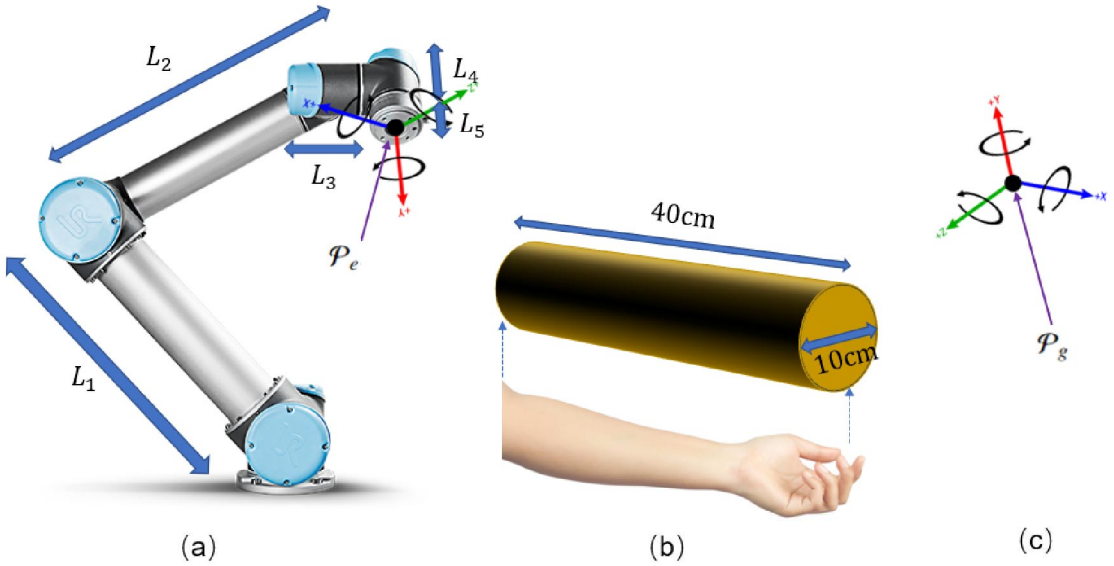


Figure 2. Graph illustration of the robot, moving obstacle and target (a) Robot links that may collide with other objects and the end-effector pose; (b) human forearm and hand modelled as a cylindrical obstacle (c) Target pose.

significant deviations would render the prediction of the obstacle's future state unreliable. These assumptions are crucial for the practical deployment of our model, ensuring that the robot can effectively forecast and navigate around the moving obstacle presented by the operator's arm.

After establishing the dynamic obstacle's position and velocity, the robot still lacks sufficient information for collision-free trajectory planning. This shortfall arises because static objects within the environment, such as the workbench, have not yet been factored into the planning process. To address this, we also provide the robot with the distances from each manipulator link to the surrounding static objects as known parameters. This additional information equips the robot's learning model with the capability to avoid collisions with both static and dynamic obstacles, as well as with its own links. As illustrated in Fig 2(a), the first link of the robot (the shoulder link) is stationary. Therefore, it is unnecessary to account for its distance to obstacles, and our considerations are confined to the remaining five links. The distances for these links are represented as $\mathcal{D} = (d_1, d_2, d_3, d_4, d_5)$, represents the smallest Euclidean distance between any point on the obstacle and link 1-5, respectively.

In summation, our model accounts for a state vector composed of 35 variables, which includes the end-effector pose, robot joint angles, goal pose, obstacle pose, obstacle velocity, and the distances of links to potential collision objects, denoted as $\mathcal{S} =< \mathcal{P}_e, \Theta, \mathcal{P}_g, \mathcal{P}_o, \mathcal{V}_o, \mathcal{D} >$. The significance of each symbol within this state vector is delineated in Table 1. The proactive nature of our algorithm is further evidenced by its consideration of state space variables, denoted as $\mathcal{P}_o$, $\mathcal{V}_o$, and $\mathcal{D}$. These variables are contingent upon the present state of obstacles. However, given the dynamic nature of the environment, the algorithm must anticipate future obstacle states and formulate trajectory plans accordingly.

Table 1. Details of the state variables.

Symbol	Description
$\mathcal{P}_e$	$x_e, y_e, z_e, \alpha_e, \beta_e, \gamma_e$
Θ	$\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6$
$\mathcal{P}_g$	$x_g, y_g, z_g, \alpha_g, \beta_g, \gamma_g$
$\mathcal{P}_o$	$x_o, y_o, z_o, \alpha_o, \beta_o, \gamma_o$
$\mathcal{V}_o$	$v_x, v_y, v_z, \omega_\alpha, \omega_\beta, \omega_\gamma$
$\mathcal{D}$	d_1, d_2, d_3, d_4, d_5

3.2. Soft actor-critic

The Soft Actor-Critic algorithm is a state-of-the-art deep reinforcement learning algorithm that optimises a stochastic policy in an off-policy way, while taking into account the entropy of the policy to encourage exploration [8]. SAC consist of two main components: an actor learns to map states to actions, and a critic learns to evaluate the selected actions by estimating the Q-value, which represents the expected return after taking an action in a given state and thereafter following a certain policy.

SAC introduces the concept of "soft" policy iteration, where the policy is not only evaluated and improved based on the expected return but also on the entropy of the policy. Entropy is a measure from information theory that quantifies the amount of randomness in the policy. A higher entropy means more exploration, as the policy is more random and thus tries out a wider range of actions. This is particularly useful in environments with many local optima to prevent premature con-

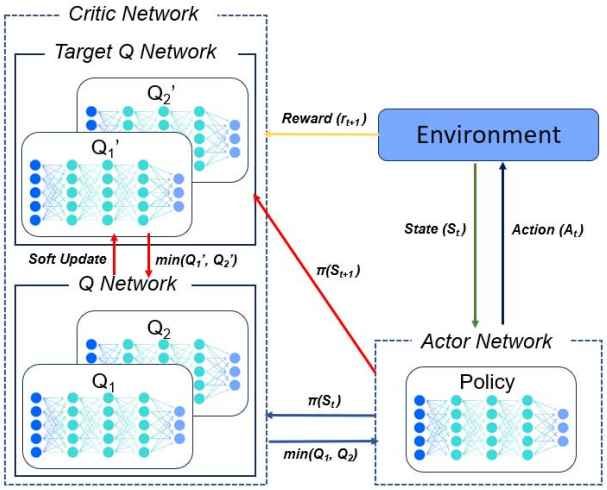


Figure 3. The network and environment interaction structure of the SAC algorithm

vergence to suboptimal policies. To address the overestimation bias of Q-values in actor-critic methods, SAC uses two Q-networks and takes the minimum of their estimated Q-values to reduce overoptimistic value estimates. In addition to the twin Q-networks, SAC also includes a separate value network that estimates the value of a state. The value function is trained to minimise the mean-squared error between its predictions and the target values, which are computed using the minimum Q-value from the twin Q-networks adjusted by the entropy term. The details of SAC algorithm is shown in Fig 3

Given the distinctive advantages over other Deep Reinforcement Learning algorithms, we selected the Soft Actor-Critic algorithm to train our model. The first advantage is its robust stability. SAC models are tempered by an entropy regularization term throughout the training process, promoting extensive exploration of the state space. This results in a stable and robust model–an essential quality for ensuring safety in human-robot collaborative manufacturing. The second advantage is sample efficiency. As an off-policy algorithm, SAC capitalises on historical data collected from previous environmental interactions. This attribute is particularly beneficial for our application, as the exploration space for a 6-DOF robot is expansive, and efficiently navigating this space is crucial for effective learning. Lastly, SAC demonstrates exceptional performance within environments characterised by continuous action spaces. This aligns perfectly with our specific setting, which involves a six-joint robot navigating around dynamic obstacles. The synergy between SAC's capabilities and our environment's demands suggests that the SAC algorithm is ideally suited to our study's requirements.

3.3. Sim-to-real transfer

A pivotal challenge in deploying deep reinforcement learning algorithms is the sim-to-real transfer, also known as domain adaptation. This entails the successful migration of a policy trained in a simulated environment to the real world. While simulation offers a safe, controllable, and cost-effective training ground, discrepancies between the simulation and real-world scenarios—collectively known as the "reality gap"—can severely hinder the performance and reliability of a robot when executing tasks such as obstacle avoidance. The reality gap encompasses differences in dynamics, perception, and environmental interactions between the simulated and physical worlds.

In our study, we propose a dual approach to minimise the reality gap: through meticulous system identification and the implementation of precision position commanding. In the system identification process, we construct a detailed model of the actual environment, which encompasses both the UR5e robot and the workbench. For the simulated counterpart, we employ the exact official Unified Robot Description Format (URDF) file used by the real robot to enhance the fidelity of the simulation. This file accurately specifies the physical parameters of the robotic arm, including the maximum speed, maximum acceleration, and rotational limits of each joint, as well as the momentum and center of mass for each link. This careful calibration of simulation parameters to mirror real-world conditions serves to significantly bridge the reality gap.

In addition to system identification, we introduce precision position commanding as a method to exert direct control over the robot's joint positions, instead of using joint velocity or torque control. Given that simulations can only approximate physical properties such as friction, mass, and inertia, torque or speed control could result in imprecise outcomes. By contrast, our model exerts direct control over the angular displacement of each joint. Coupled with the insights gained from system identification, this approach ensures that the robot's state post-execution aligns precisely with real-world behaviour. Figure 4 illustrates our training and deployment framework, highlighting the strategies we employ to minimise the reality gap. The selection of PyBullet as our physics engine stems from its seamless integration with Python, our primary programming language within the development environment. This compatibility facilitates smoother integration and implementation processes. Practically speaking, PyBullet enables us to import the official URDF file directly, thereby enabling the construction of a simulated robot with identical physical properties to its real-world counterpart.

4. Reward Function Design

The cornerstone of our study lies in the innovative design of the reward function. While prior research has explored the training of robotic manipulators using RL and DRL, to our knowledge, our approach is the first to tackle the challenge of avoiding dynamically moving obstacles in a 3D space solely through DRL, without relying on supplementary algorithms such as the potential field method. Our distinctive reward design is instrumental in directing the model to accomplish its objectives using a vanilla DRL algorithm. This not only simplifies the learning process but also facilitates the transferability of our method to different robotic platforms and environments.

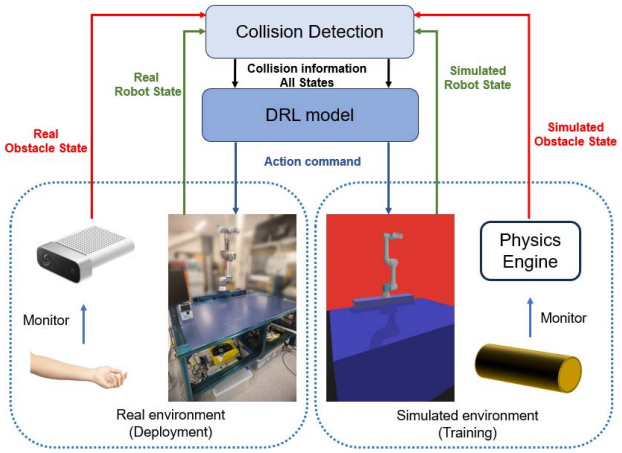


Figure 4. The framework for training and deploying the DRL model, minimises the reality gap.

Our goal is to navigate the robot's end-effector from an initial pose $\mathcal{P}_e$ to a target pose $\mathcal{P}_g$, controlling the joint angles Θ without any collision with free-moving obstacles. To this end, our reward function is composed of five components:

Collision Reward r_c: Penalises the robot if the end-effector collides with anything, ensuring safe manoeuvring. The algorithm determines Whether a collision occurred based on the distances between robot links and other objects $\mathcal{D}$. If any of these values is equal to or smaller than zero, it indicates the manipulator collides, the training episode immediately stops and penalises the model with a -500 reward. Equation for collision reward is given in Eqn (3).

$$r_c = \begin{cases} -500 & \text{if collision} \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

Success Reward r_s: Rewards the robot for successfully reaching the target pose, reinforcing the completion of the task. If the manipulator successfully reaches the target, the training episode also immediately stops and rewards the model with a 200 reward, given in Eqn (4).

$$r_s = \begin{cases} 200 & \text{if success} \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

Positional Reward r_p: Provides incremental feedback based on the proximity of the end-effector's current position p_e to the goal position p_g, guiding the trajectory. The positional reward is calculated by the Euclidean distance between two points with a scale of -70, given in Eqn (5)(6).

$$\text{Positional distance} = \sqrt{(x_e - x_g)^2 + (y_e - y_g)^2 + (z_e - z_g)^2} \tag{5}$$

$$r_p = \text{Positional distance} \times (-70) \tag{6}$$

Rotational Reward r_r: Encourages the correct orientation of the end-effector, aligning it with the desired goal orientation. Since the the end-effector and target rotation in the model states are in Euler angle format, we need first to convert them to quaternions: $(\alpha_e, \beta_e, \gamma_e)$ (i_e, j_e, k_e, w_e), $(\alpha_g, \beta_g, \gamma_g)$ (i_g, j_g, k_g, w_g). Then the rotational reward is calculated by the smallest distance between two quaternions with a scale of -30, given in Eqn (7)(8).

$$\text{Rotational distance} = 2 \times \arccos\left(\left|i_e i_g + j_e j_g + k_e k_g + w_e w_g\right|\right) \tag{7}$$

$$r_r = \text{Rotational distance} \times (-30) \tag{8}$$

Distance Reward r_d: Penalises moving towards the obstacle, ensuring safe distance while moving. For every moveable link on the robot, it is penalised for moving closer to the obstacle and rewarded for moving away from the obstacle when the link distance is smaller than 0.2 meters. If the link distance is bigger than 0.2 meters, it is considered safe and the reward is 0.

$$r_d = \sum_{n=1}^{5} \begin{cases} \text{weight}_i \times \Delta\text{distance}_i & \text{if distance}_i < 0.2 \\ 0 & \text{otherwise} \end{cases} \tag{9}$$

The weight of each link in the distance reward is different in our work, determined by the total link distance reward weight and each link's physical weight. From link 1 to link 5, their physical weights are approximately 8, 2.4, 1.2, 1.2, 0.2 Kilograms. The total link distance reward weight is set to 50, and the detailed reward weight for each link is given in Eqn (10). It needs to be noticed that the link weights are set to encourage the algorithm to optimize for energy efficiency by preferring movements that involve lighter links over heavier ones, whenever possible. They do not perfectly reflect the true dynamics of the robotic arm.

$$\begin{aligned} \text{weight}_i &= \frac{\text{link physical weights}}{\sum \text{link physical weights}} \times 50 \\ &= [30.77, 9.23, 4.62, 4.62, 0.77] \end{aligned} \tag{10}$$

In our reinforcement learning model, we tuned the reward weights for positional and rotational distance through a systematic trial-and-error process. We initially set these weights to a total of -100. This baseline allowed us to adjust the weights for collision and success in subsequent trials. Determining the optimal balance between positional and rotational rewards was challenging, as it influences whether the model prioritizes

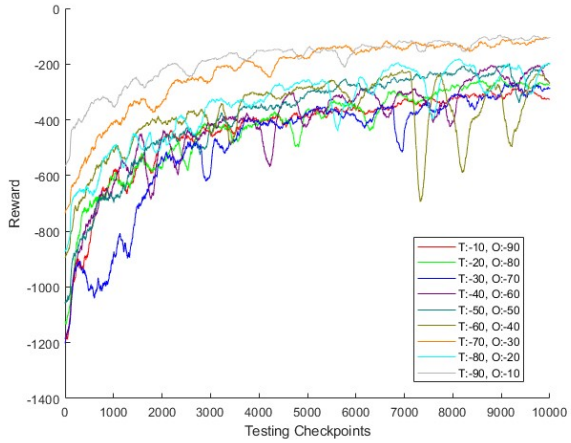


Figure 5. Nine reward curves for different combinations of positional and rotational reward weight.

achieving target positions or rotations. We designed an experiment with nine different weight scenarios, varying positional rewards from -10 to -90, while keeping other parameters constant. After training each scenario for one million episodes, we illustrated the resulting reward curves in Figure 5, where each curve correlates with a distinct weight setting. For example, the line labeled "T:-20, O:-80" represents the scenario with a positional weight of -20 and a rotational weight of -80. Our analysis indicated that a positional reward weight of -70 and a rotational weight of -30 yielded the most balanced and effective learning outcomes.

Setting the weights for collision and success was less complex. Based on the total weight of -100 for positional and rotational distances, we selected -500 for collision and +200 for success after simple preliminary testing. The final hyperparameter, the total link distance weight, affects the manipulator's behaviour regarding obstacles. A weight that is too high results in excessive avoidance behaviour, preventing the manipulator from reaching its target pose. Conversely, a weight that is too low increases the risk of collision. After further experimentation, we determined that a weight of 50 struck the right balance. To provide a clear understanding of our approach, we have outlined the reward calculation process in the accompanying pseudocode Algorithm 1.

Algorithm 1 Compute Reward

for each *step* in *episode* **do**
  if *step is* 0 **then**
    $previous_dist \leftarrow 0$
    $reward \leftarrow 0$
  end if
  if *collision* **then**
    return $collision_weight$
  end if
  if *success* **then**
    return $success_weight$
  end if
  $link_dist \leftarrow get_current_link_dist$
  $\Delta link_dist \leftarrow link_dist - previous_dist$
  $previous_dist \leftarrow link_dist$
  $pos_dist \leftarrow calc_pos_dist(current_pose, target_pose)$
  $rot_dist \leftarrow calc_rot_dist(current_pose, target_pose)$
  $reward \leftarrow reward + positional_weight \cdot pos_dist$
  $reward \leftarrow reward + rotational_weight \cdot rot_dist$
  for each $\Delta dist_i$ in $\Delta link_dist$ **do**
    if $\Delta dist_i < 0.2$ **then**
      $reward \leftarrow reward + link_dist_weight_i \cdot \Delta dist_i$
    end if
  end for
end for
return *reward*

5. Experiment

In this section, we begin by detailing our experiment setup in a simulated environment, outlining our model's training and testing procedures. Subsequent analysis of the training results and test data set will follow, proofing the performance of our method.

5.1. Training and testing setup

In our SAC algorithm training, PyBullet was key in creating a simulated environment mirroring our robotic setup. Renowned for its effectiveness in simulations, robotics, and machine learning, PyBullet enabled a training environment that surpassed the limitations of real-world settings. This approach not only sped up the training process but also prevented potential harm to both the robot and humans from unexpected movements during training.

Fig 6 visualises our simulation setup. It features the UR5e robot positioned on a linear track installed on the workbench. While the track enhances the robot's operational range, it was not employed in this study. Our setup includes a multicoloured cube, serving as a visual guide for the target pose. The smaller, transparent cuboid below represents the target position sampling area. Arrows at the top left corner indicate the range for sampling target rotations, with the robot's end-effector designed to avoid pointing upwards or backward, aligning with practical constraints in real-world robot grasping. When the robot is reaching for the target pose, we consider the action successful when the positional distance is within 3cm and the rotational distance is within 5 degrees. The yellow cylinder in the simulation represents the moving obstacle, symbolising a human hand and forearm; its darker end represents the hand, and the lighter end, the elbow. The larger transparent cuboid above delineates the moving obstacle's sampling range.

Each episode begins with a randomly selected target position and rotation. The starting and ending poses for the obstacle are also randomly chosen. Throughout the episode, this obstacle moves and rotates at a constant speed from its starting to end position over two seconds, then remains stationary. Training examples are generated through direct interaction between

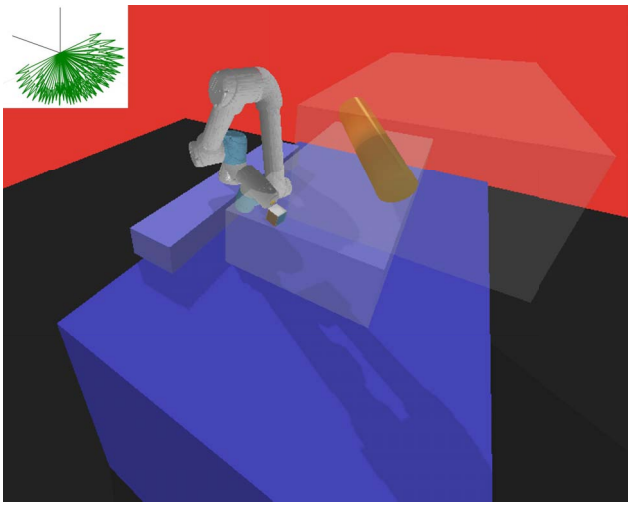


Figure 6. Our simulation setup, target and obstacle sampling range

Table 2. List of training hyperparameters.

Hyperparameter	Value
Episodes	4000000
Steps per Episode	100
Buffer Size	10000000
Learning Rate	0.0001
Gamma	0.95
Batch Size	256

the robot and the environment. At each step of this interaction, the robot executes actions, consequently inducing changes in the environment's state. The volume of our training data hinges on two primary factors: the number of episodes and the number of steps per episode. We chose a significantly large number for the training episodes to encompass all possible positions of the human arm during interaction with the manipulator in the large operational 3D space. This extensive coverage is critical for preparing our model to handle a diverse array of real-world scenarios. Thereby fostering robustness, adaptability and generalizability to a broad spectrum of human-arm and robotic-manipulator coexistence scenarios. Additional hyperparameters fine-tuned to optimise training results are detailed in Table 2.

5.2. *Model validation and test result*

During the training process, it's crucial to periodically validate the model and save it if it surpasses previous performance levels. As outlined in Table 2, the training encompasses 4 million episodes. We validate the model after every 1,000 training episodes using a 100-episode validation set, resulting in 4,000 data points. The model is saved when it achieves a higher success rate than before. In Fig 7, we display the mean reward, mean episode length, and success rate for these 4,000 data points.

Initially, the mean reward hovers around -700, indicating frequent collisions by the robot with either the obstacle or itself. This leads to a short mean episode length and a consistent success rate of zero. However, after approximately 300 validation points, there is a notable shift: the reward drops, and the mean episode length jumps to around 80 steps. This suggests the robot has started to learn collision avoidance, though it has not yet mastered reaching its goal. As training progresses, the mean reward gradually improves, eventually stabilising at about -80. Correspondingly, the mean episode length decreases and levels out at around 10 steps. The success rate also shows significant improvement, rising from zero and stabilising above 90%. This progression indicates that the robot is not only learning to achieve its target but is also becoming more efficient in doing so, minimising the time taken to complete tasks.

Even though we have validation episodes during the training, the size of it is limited to 100, and the goal position and orientation are totally random. It cannot represent the real performance of our trained model. In response, we devised a test set to evaluate the trained model. As shown in Fig 6, the target position is sampled within a cuboid area with 1.0m, 0.45m, and 0.2m in length, width, and height, respectively. We used a uniform sampling strategy wherein a target position is sampled every 0.05m within the goal area, and for each of these target positions, five random orientations are sampled. This methodology resulted in a total of 5250 points ($10 \times 21 \times 5 \times 5 = 5250$) within our test set. Upon evaluation of our trained model against this test set, we secured a success rate of 92.97%. We prioritize the success rate as the primary performance metric based on its ability to offer a clear and intuitive measure of effectiveness across a large number of testing episodes. While metrics like minimum distance to obstacles and episode completion time do provide meaningful insights on a per-episode level, their value diminishes when averaged over a substantial volume of testing episodes. This is because averaging these metrics can obscure important nuances and variations in performance that are essential for a comprehensive evaluation of safety and effectiveness. For instance, outliers in minimum distance or completion time can significantly affect their average values, potentially leading to misleading interpretations of the algorithm's overall performance.

Although our model has been tested using a generated test dataset, we cannot conclusively assert that it has effectively learned obstacle avoidance. This uncertainty arises because the model achieved a 93% success rate, yet the obstacles' starting and ending positions were randomly generated. Consequently, it is plausible that the model's success was due to chance rather than genuine learning. To address this, we trained an additional model in an environment without obstacles (red dash lines in Fig 8) and subsequently tested it on the same dataset. We compared the end-effector trajectories from both models in the same test scenario, alongside the obstacle's trajectory, as illustrated in Fig 8. This comparison distinctly demonstrates that the model trained with dynamic obstacle avoidance consistently maintains a safe distance from a moving obstacle, whereas the model lacking this training collided with the obstacle

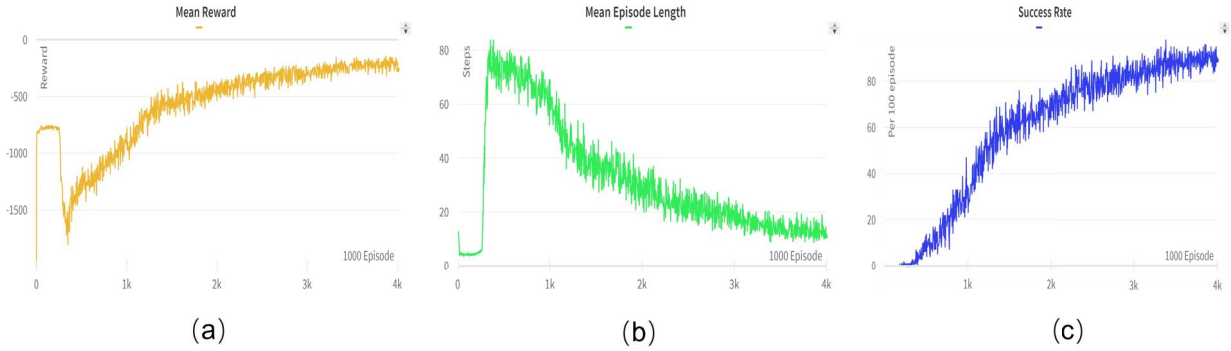


Figure 7. Model validation mean results for 100 episodes (a) Mean reward; (b) Mean episode length; (c) Success rate.

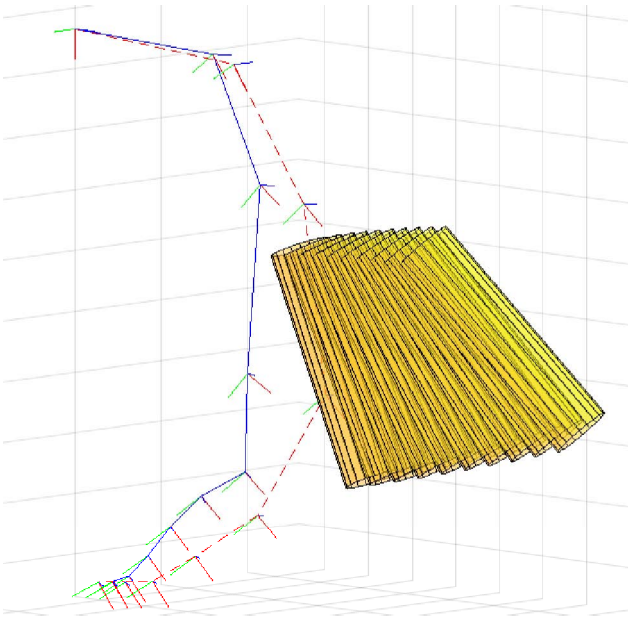


Figure 8. The robot end-effector trajectory obtained from two models on the same testing case. Yellow: The moving trajectory of the obstacle. Blue: The waypoints generated by the model trained with moving obstacles. Red: The waypoints generated by the model trained without obstacles.

6. Conclusion and Future Work

This research presents a significant advancement in the field of human-robot collaboration, offering a novel deep reinforcement learning-based approach for dynamic obstacle avoidance. The use of the SAC algorithm, combined with a well-crafted reward function, has proven effective in enabling a robotic manipulator to navigate around moving human obstacles safely and efficiently.

As we look to the future, we plan to finish the real-world experiment of our proposed system as shown in Fig 4. Our strategy involves utilizing open-source human skeleton motion recognition algorithms to process the data captured by the RGB-D camera. This will allow us to extract detailed skeletal motion, including the precise movements and positioning of the human forearm. Concurrently, we can obtain real-time poses of the robot's links through feedback from the robot's status systems. By integrating these two data sources, we will be able to calculate the distances between each robot link and the human arm with high precision.

In conclusion, this study not only addresses a critical need in the realm of human-robot collaboration but also lays the groundwork for future advancements in robotics and automated systems, emphasizing safety, efficiency, and adaptability.

References

[1] Casalino, A., Bazzi, D., Zanchettin, A.M., Rocco, P., 2019. Optimal proactive path planning for collaborative robots in industrial contexts, in: 2019 International Conference on Robotics and Automation (ICRA), IEEE. pp. 6540–6546.

[2] Chen, L., Jiang, Z., Cheng, L., Knoll, A.C., Zhou, M., 2022a. Deep reinforcement learning based trajectory planning under uncertain constraints. Frontiers in Neurorobotics 16, 883562.

[3] Chen, P., Pei, J., Lu, W., Li, M., 2022b. A deep reinforcement learning based method for real-time path planning and dynamic obstacle avoidance. Neurocomputing 497, 64–75.

[4] Cheng, X., Liu, S., 2022. Dynamic obstacle avoidance algorithm for robot arm based on deep reinforcement learning, in: 2022 IEEE 11th Data Driven Control and Learning Systems Conference (DDCLS), IEEE. pp. 1136–1141.

[5] El-Shamouty, M., Wu, X., Yang, S., Albus, M., Huber, M.F., 2020. Towards safe human-robot collaboration using deep reinforcement learning, in: 2020 IEEE international conference on robotics and automation (ICRA), IEEE. pp. 4899–4905.

[6] Fujimoto, S., Hoof, H., Meger, D., 2018. Addressing function approximation error in actor-critic methods, in: International conference on machine learning, PMLR. pp. 1587–1596.

[7] Gu, S., Lillicrap, T., Sutskever, I., Levine, S., 2016. Continuous deep q-learning with model-based acceleration, in: International conference on machine learning, PMLR. pp. 2829–2838.

[8] Haarnoja, T., Zhou, A., Abbeel, P., Levine, S., 2018. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: International conference on machine learning, PMLR. pp. 1861–1870.

[9] Haddadin, S., Urbanek, H., Parusel, S., Burschka, D., Roßmann, J., Albu-Schäffer, A., Hirzinger, G., 2010. Real-time reactive motion generation based on variable attractor dynamics and shaped velocities, in: 2010 IEEE/RSJ International Conference on Intelligent Robots and Systems, IEEE. pp. 3109–3116.

[10] Hadfield-Menell, D., Milli, S., Abbeel, P., Russell, S.J., Dragan, A., 2017. Inverse reward design. Advances in neural information processing systems 30.

[11] Ibarz, J., Tan, J., Finn, C., Kalakrishnan, M., Pastor, P., Levine, S., 2021. How to train your robot with deep reinforcement learning: lessons we have learned. The International Journal of Robotics Research 40, 698–721.
[12] Lasota, P., Nikolaidis, S., Shah, J., 2013. Developing an adaptive robotic assistant for close proximity human-robot collaboration in space. AIAA Infotech at Aerospace (I at A) Conference doi:10.2514/6.2013-4806.
[13] Lasota, P.A., Shah, J.A., 2015. Analyzing the effects of human-aware motion planning on close-proximity human–robot collaboration. Human factors 57, 21–33.
[14] Lee, Y., Hu, E.S., Lim, J.J., 2021. Ikea furniture assembly environment for long-horizon complex manipulation tasks, in: 2021 ieee international conference on robotics and automation (icra), IEEE. pp. 6343–6349.
[15] Li, H., Gong, D., Yu, J., 2021. An obstacles avoidance method for serial manipulator based on reinforcement learning and artificial potential field. International Journal of Intelligent Robotics and Applications 5, 186–202.
[16] Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D., 2015. Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971 .
[17] Liu, Q., Liu, Z., Xiong, B., Xu, W., Liu, Y., 2021. Deep reinforcement learning-based safe interaction for industrial human-robot collaboration using intrinsic reward function. Advanced Engineering Informatics 49, 101360.
[18] Lo, S.Y., Cheng, C.A., Huang, H.P., 2016. Virtual Impedance Control for Safe Human-Robot Interaction. Journal of Intelligent and Robotic Systems: Theory and Applications 82, 3–19. doi:10.1007/s10846-015-0250-y.
[19] Matheson, E., Minto, R., Zampieri, E.G., Faccio, M., Rosati, G., 2019. Human–robot collaboration in manufacturing applications: A review. Robotics 8, 100.
[20] Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., Kavukcuoglu, K., 2016. Asynchronous methods for deep reinforcement learning, in: International conference on machine learning, PMLR. pp. 1928–1937.
[21] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., Riedmiller, M., 2013. Playing atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602 .
[22] Polverini, M.P., Zanchettin, A.M., Rocco, P., 2014. Real-time collision avoidance in human-robot interaction based on kinetostatic safety field, in: 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems, IEEE. pp. 4136–4141.
[23] Ragaglia, M., Zanchettin, A.M., Rocco, P., 2018. Trajectory generation algorithm for safe human-robot collaboration based on multiple depth sensor measurements. Mechatronics 55, 267–281.
[24] Robla-Gómez, S., Becerra, V.M., Llata, J.R., Gonzalez-Sarabia, E., Torre-Ferrero, C., Perez-Oria, J., 2017. Working together: A review on safe human-robot collaboration in industrial environments. Ieee Access 5, 26754–26773.
[25] Rubagotti, M., Taunyazov, T., Omarali, B., Shintemirov, A., 2019. Semi-Autonomous Robot Teleoperation with Obstacle Avoidance via Model Predictive Control. IEEE Robotics and Automation Letters 4, 2746–2753. doi:10.1109/LRA.2019.2917707.
[26] Sangiovanni, B., Rendiniello, A., Incremona, G.P., Ferrara, A., Piastra, M., 2018. Deep reinforcement learning for collision avoidance of robotic manipulators, in: 2018 European Control Conference (ECC), IEEE. pp. 2063–2068.
[27] Schulman, J., Levine, S., Abbeel, P., Jordan, M., Moritz, P., 2015. Trust region policy optimization, in: International conference on machine learning, PMLR. pp. 1889–1897.
[28] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347 .
[29] Sutton, R.S., Barto, A.G., 2018. Reinforcement learning: An introduction. MIT press.
[30] Villani, V., Pini, F., Leali, F., Secchi, C., 2018. Survey on human–robot collaboration in industrial settings: Safety, intuitive interfaces and applications. Mechatronics 55, 248–266.
[31] Wu, K., Esfahani, M.A., Yuan, S., Wang, H., 2019. Depth-based obstacle avoidance through deep reinforcement learning, in: Proceedings of the 5th International Conference on Mechatronics and Robotics Engineering, pp. 102–106.
[32] Xia, W., Lu, Y., Xu, X., 2023. Application and evaluation of soft-actor critic reinforcement learning in constrained trajectory planning for 6dof robotic manipulators, in: 2023 29th International Conference on Mechatronics and Machine Vision in Practice (M2VIP), IEEE. pp. 1–6.
[33] Zindler, F., Lucchi, M., Wohlhart, L., Pichler, H., Hofbaur, M., 2022. Towards dynamic obstacle avoidance for robot manipulators with deep reinforcement learning, in: International Conference on Robotics in Alpe-Adria Danube Region, Springer. pp. 89–96.